



• Dar Lila

Guide du projet

Boutique en ligne — parfums, cosmétiques & articles bébé

Laravel 12 · API REST

Vue 3 · Vite · Pinia

MySQL

Sanctum

Bunny CDN

FR / AR · RTL

Architecture, comptes, URLs et procédures d'exploitation
de la boutique développée avec Laravel & Vue.js

Version 1.0 — Août 2026

Refonte du site arsenaldza-coif.github.io/boutique1

◆ 1 • Présentation du projet

Ce que fait la boutique, et ce qui la distingue.

Dar Lila est une boutique en ligne algérienne (parfums, cosmétiques, articles bébé) refondue à partir du site statique d'origine en une application moderne **Laravel + Vue.js**, avec une gestion commerciale complète et la commande finalisée via **WhatsApp**.

Boutique publique

- Catalogue de **22 produits** répartis en **8 catégories**, bilingue français / arabe (RTL instantané)
- Filtres par catégories, recherche instantanée FR/AR, fiches produit en modale
- Panier persistant (localStorage) avec recommandations produits
- Commande en 3 étapes : panier → coordonnées (58 wilayas) → confirmation
- Commande **enregistrée en base** (prix recalculés côté serveur) puis récapitulatif **pré-rempli sur WhatsApp** avec référence **DL-XXXXXX**
- Galerie photos avec visionneuse, avis clients, carte Google Maps, design fidèle à l'original (ivoire / or / indigo, séparateurs zellige)

Espace d'administration (/admin)

- Authentification **Laravel Sanctum** (tokens Bearer, anti brute-force)
- CRUD produits complet : noms FR/AR, prix, catégorie, slug auto, image par **URL ou téléversement direct vers Bunny Storage**, mise en vedette, activation
- Suivi des **commandes** : statistiques, filtres, détail dépliable, statuts (nouvelle → confirmée → expédiée → livrée / annulée)

Services intégrés

- **Notification e-mail au vendeur** à chaque commande (Markdown, file d'attente, repli sûr)
- **Mode maintenance** : page d'attente élégante, admin préservé, retour automatique du site
- **CDN Bunny.net** : cache des images + stockage direct depuis l'admin
- **Outil de déploiement** SFTP/SSH vers Hostinger en une commande

Qualité vérifiée

L'ensemble a été testé automatiquement (Playwright) : parcours d'achat complet, CRUD admin, uploads, bascule AR/RTL, mode maintenance, protection des routes — zéro erreur console.

Chiffres clés

77 fichiers backend Laravel	45 fichiers frontend Vue
7 tables SQL + 2 tables auth	15 composants + 4 vues admin
15 endpoints API	2 langues (FR / AR + RTL)



◆ 2 • Architecture

Une SPA Vue 3 consommant une API REST Laravel — données MySQL, médias Bunny, messagerie WhatsApp.

◆ **Navigateur client**
SPA Vue 3 + Vite + Pinia + Vue Router
Fichiers statiques (Hostinger / Bunny CDN)

◆ **Navigateur vendeur**
Espace admin `/admin` de la même SPA
Token Sanctum dans localStorage

▼ fetch JSON (HTTPS) ▼

◆ **API REST — Laravel 12**
Routes `/api/*` · contrôleurs + Ressources `{data:...}` · validation 422
Auth Sanctum (Bearer) · middleware `admin` · mode maintenance maison



◆ **MySQL**
produits · catégories · commandes · avis · galerie · réglages · users · tokens

◆ **Bunny.net**
CDN (\$0.01/Go) + Storage (upload admin) + Stream (vidéos)

◆ **Sorties externes**
wa.me (commandes WhatsApp) · SMTP Hostinger (e-mail vendeur)

Flux d'une commande
1. Le client remplit le panier → 2. `POST /api/orders` (prix recalculés serveur, référence générée) → 3. e-mail automatique au vendeur → 4. WhatsApp s'ouvre avec le récapitulatif pré-rempli → 5. le vendeur suit la commande dans `/admin/orders` .

Sécurité intégrée

MESURE	où
Prix & totaux recalculés côté serveur	<code>OrderController::store</code>
Validation stricte (Laravel FormRequest) → 422 + erreurs par champ	toutes les écritures
Routes admin protégées <code>auth:sanctum</code> + middleware <code>admin</code>	<code>routes/api.php</code>
Limitation des tentatives de connexion (6/min)	<code>/api/auth/login</code>
Mots de passe hachés (bcrypt), jamais renvoyés par l'API	<code>User::toSafeArray</code>
CORS restreint au frontend déclaré	<code>config/cors.php</code> + <code>.env</code>

◆ 3 • Structure des dossiers

Quatre briques indépendantes dans le dépôt `darlila/`.

```
darlila/ └─ backend-laravel/ - API REST (PHP ≥ 8.2) | └─ app/ | | └─ Console/Commands/ shop:down · shop:up  
(maintenance) | | └─ Http/Controllers/ catalogue · commande · auth | | | └─ Admin/ produits · commandes ·  
config | | └─ Http/Middleware/ EnsureUserIsAdmin · ShopMaintenance | | └─ Http/Resources/ formatage JSON  
{data:...} | | └─ Models/ Product, Order, User, Setting... | | └─ Notifications/ OrderPlaced (e-mail vendeur) |  
| └─ Services/ BunnyStorage (API Bunny) | └─ database/ | | └─ migrations/ 9 tables | | └─ seeders/ 22  
produits FR/AR, admin, réglages | └─ resources/views/errors/503.blade.php page maintenance HTML | └─  
routes/api.php tous les endpoints | └─ public/index.php compatible public_html (mutualisé) | └─ frontend-  
vue/ - SPA (Node ≥ 18) | └─ src/ | └─ views/ HomeView · Admin (login/produits/commandes) | └─ components/  
boutique + admin/ (layout, formulaire produit) | └─ stores/ shop · cart · ui · admin (Pinia) | └─  
api/client.js fetch + token Bearer + multipart | └─ router/ routes + garde d'authentification | └─ i18n/ FR  
/ AR + bascule RTL | └─ utils/ whatsapp · wilayas (58) | └─ api-mock/ - simulation Node de l'API (zéro  
dépendance) | └─ server.js mêmes endpoints + auth + upload + maintenance | └─ deploy/ - outil de déploiement  
Hostinger | └─ deploy.mjs build + SFTP + artisan en une commande | └─ deploy.config.example.json à copier →  
deploy.config.json (secrets)
```

À ne pas versionner

`.env` (secrets), `vendor/`, `node_modules/`, `database.sqlite`, `deploy/deploy.config.json` — tous déjà couverts par les `.gitignore` du projet.

Démarrer sans PHP : le mode simulation

`node api-mock/server.js` reproduit fidèlement l'API Laravel (auth Sanctum, admin, uploads, maintenance) avec les mêmes données — pratique pour développer le frontend ou faire une démo.

◆ 4 • Comptes & accès

Tous les identifiants par défaut — à personnaliser avant la mise en production.

Comptes par défaut (développement)

ÉLÉMENT	IDENTIFIANT	MOT DE PASSE / VALEUR	MODIFIER OÙ
Admin boutique	<code>admin@darlila.dz</code>	<code>password</code>	<code>.env</code> → <code>ADMIN_EMAIL</code> / <code>ADMIN_PASSWORD</code> <i>avant le seed</i>
Base MySQL	<code>root</code> (local)	(vide)	<code>.env</code> → <code>DB_*</code>
WhatsApp boutique	<code>+213 555 00 00 00</code>	<code>213555000000</code>	<code>.env</code> → <code>WHATSAPP_NUMBER</code>
E-mail vendeur	<code>admin@darlila.dz</code>	—	<code>.env</code> → <code>SELLER_EMAIL</code>
SMTP (prod)	mailbox Hostinger	à créer	<code>.env</code> → <code>MAIL_*</code> (smtp.hostinger.com:465)
Bunny Storage	zone + clé API	à créer	<code>.env</code> → <code>BUNNY_STORAGE_*</code>

Avant l'ouverture au public

Changez **ADMIN_PASSWORD**, régénérez **APP_KEY**, passez **APP_DEBUG=false** et activez **HTTPS**. Voir la checklist page 9.

URLs en développement local

SERVICE	URL	LANCEMENT
Boutique (SPA)	<code>http://localhost:5173</code>	<code>cd frontend-vue && npm run dev</code>
API Laravel	<code>http://127.0.0.1:8000/api</code>	<code>cd backend-laravel && php artisan serve</code>
API simulée (sans PHP)	<code>http://127.0.0.1:8000/api</code>	<code>node api-mock/server.js</code>
Administration	<code>http://localhost:5173/admin</code>	—

En dev, Vite proxifie `/api` et `/storage` vers le port 8000 (cible modifiable via `API_PROXY_TARGET`).

URLs en production (exemple)

SERVICE	URL	HÉBERGEMENT
Boutique publique	<code>https://darlila.com</code>	Hostinger — <code>public_html</code> du domaine
API Laravel	<code>https://api.darlila.com</code>	Hostinger — sous-domaine dédié
Administration	<code>https://darlila.com/admin</code>	—
Images CDN	<code>https://darlila.b-cdn.net/...</code>	Bunny.net

◆ 5 • Endpoints de l'API

Format commun : `{ "data": ... }` en succès · `{ "message", "errors" }` en erreur 422.

Publics

MÉTHODE	URI	DESCRIPTION
GET	/api/settings	Réglages publics (WhatsApp, adresse, horaires, hero)
GET	/api/categories	8 catégories + nombre de produits
GET	/api/products	Produits actifs — <code>?category=&search=&featured=1</code>
GET	/api/products/{slug}	Détail produit (404 si inconnu)
GET	/api/reviews · /api/gallery	Avis clients · galerie
POST	/api/orders	Enregistre une commande → référence + lien WhatsApp + e-mail vendeur

Authentification (Sanctum)

MÉTHODE	URI	DESCRIPTION
POST	/api/auth/login	<code>{email, password}</code> → <code>{token, user}</code> (throttle 6/min)
GET	/api/auth/me	Utilisateur courant — <code>Authorization: Bearer ...</code>
POST	/api/auth/logout	Révoque le token

Administration (token requis)

MÉTHODE	URI	DESCRIPTION
GET	/api/admin/config	Capacités (stockage images : bunny / local)
GET	/api/admin/categories	Catégories (disponibles même en maintenance)
GET	/api/admin/products	Tous produits (actifs + inactifs)
POST	/api/admin/products	Création — JSON avec <code>image_url</code> ou multipart fichier <code>image</code>
PUT	/api/admin/products/{id}	Mise à jour complète ou partielle (<code>{is_active:false}</code>)
DEL	/api/admin/products/{id}	Suppression (+ nettoyage image Bunny)
GET	/api/admin/orders	Commandes + articles — <code>?status=</code>
PATCH	/api/admin/orders/{id}/status	Changement de statut

Exemple — passer commande

```
curl -X POST https://api.darlila.com/api/orders \
-H "Content-Type: application/json" \
-d '{"customer_name":"Sarah Benali","customer_phone":"0555123456", "wilaya":"16 - Alger","items": [{"product_id":9,"quantity":2}]}' → 201 {"data":{"reference":"DL-4A5EE7","total":13600,...}}
```

◆ 6 • Procédures courantes

Installation, lancement, gestion quotidienne.

Installer le projet (local, MySQL)

```
# 1. Base de données (une fois) mysql -u root -p -e "CREATE DATABASE darlila CHARACTER SET utf8mb4;" # 2. Backend cd backend-laravel composer install cp .env.example .env # ajuster DB_* si besoin php artisan key:generate composer setup # migrations + seeders + storage:link # 3. Frontend (autre terminal) cd ../frontend-vue && npm install && npm run dev
```

Gérer la boutique au quotidien (vendeur)

- **Ajouter un produit** : `/admin` → Produits → « + Nouveau produit » → remplir FR/AR, prix, catégorie, image (URL ou fichier → Bunny si configuré)
- **Masquer un produit** : interrupteur « Actif » dans la liste — il disparaît immédiatement de la boutique
- **Traiter une commande** : `/admin/orders` → consulter le détail, appeler le client (paiement à la livraison), faire évoluer le statut
- **E-mails** : chaque commande envoie une notification à `SELLER_EMAIL` (avec `MAIL_MAILER=log`, les messages sont dans `storage/logs/laravel.log`)

Mettre la boutique en maintenance

```
# Sur le serveur (backend) : php artisan shop:down --message="Retour à 14h" --retry=600 # → page d'attente bilingue, admin PRÉSERVÉ, auto-retour dès shop:up php artisan shop:up # rouvrir la boutique
```

La page d'attente vérifie toutes les 20 secondes si la boutique est revenue et se recharge automatiquement. Un bouton WhatsApp permet au client de commander malgré tout.

Après toute modification du code

```
cd deploy && npm run deploy # rebuild + upload + migrations + caches
```

◆ 7 · Déploiement — Hostinger + Bunny.net

Mise en ligne complète en une commande grâce à l'outil `deploy/`.

Préparer Hostinger (une fois)

- hPanel → **Bases de données MySQL** : créer base + utilisateur
- hPanel → **Avancé** → **Accès SSH** : noter hôte / port / identifiants
- Créer le **sous-domaine** `api.votre-domaine.com` (il hébergera Laravel)
- Choisir le datacenter **France (Paris)** pour la meilleure latence vers l'Algérie

Configurer et lancer

```
cd deploy npm install # client SSH/SFTP cp deploy.config.example.json deploy.config.json # vos accès + MySQL + SMTP npm run dry-run # simulation : build + plan, rien n'est envoyé npm run deploy # déploiement réel
```

L'outil enchaîne : build du frontend (avec l'URL d'API de production) → upload SFTP du backend (`public/` renommé `public_html/`, `index.php` compatible mutualisé) → upload du frontend + `.htaccess` SPA → `composer install` distant → écriture du `.env` production → migrations + seeders → `storage:link` + `php artisan optimize` → vérifications HTTP.

Activer Bunny.net (images & vidéos)

NIVEAU	CONFIGURATION	USAGE
1 · Pull zone	<code>BUNNY_CDN_URL=https://...b-cdn.net</code>	Cache des images serveur (zéro code)
2 · Storage natif	<code>BUNNY_STORAGE_ZONE / _KEY / _CDN_URL</code>	Upload direct depuis l'admin → edge mondial
3 · Stream	bunny.net → Stream	Vidéos produit / hero (player + transcodage inclus)

Coûts Bunny : CDN \$0.01/Go (EU), stockage \$0.01/Go/mois, minimum \$1/mois — environ \$1-3/mois pour Dar Lila. Les visiteurs algériens sont servis depuis les edge européens.

Architecture cible

`darlila.com` → frontend statique (dist) · `api.darlila.com` → Laravel · `darlila.b-cdn.net` → médias · cross-origin géré par le CORS déjà configuré (`CORS_ALLOWED_ORIGINS`).

◆ 8 · Sécurité, exploitation & dépannage

Checklist de mise en production et réponses aux incidents courants.

Checklist avant ouverture au public

- ✓ Changer `ADMIN_PASSWORD` dans le `.env` puis `php artisan db:seed --class=UserSeeder`
- ✓ Régénérer `APP_KEY` (`php artisan key:generate`) — invalide les données chiffrées
- ✓ Vérifier `APP_ENV=production` et `APP_DEBUG=false`
- ✓ HTTPS actif sur les deux domaines (Let's Encrypt gratuit chez Hostinger)
- ✓ `CORS_ALLOWED_ORIGINS=https://darlila.com` uniquement
- ✓ SMTP configuré (`MAIL_MAILER=smtp`) et e-mail de commande reçu en test
- ✓ Sauvegardes quotidiennes Hostinger activées (incluses plan Unlimited)
- ! Ne jamais versionner `.env` ni `deploy.config.json`

Dépannage rapide

SYMPTÔME	SOLUTION
Erreur 500 après édition du <code>.env</code>	<code>php artisan config:cache</code>
Images 404 après upload local	<code>php artisan storage:link</code>
migrate échoue	Vérifier <code>DB_*</code> (hPanel → Bases de données)
Admin : « Session expirée »	Se reconnecter (token révoqué) — comportement normal
Boutique bloquée en maintenance	<code>php artisan shop:up</code>
API injoignable en production	Propagation DNS/SSL — vérifier le sous-domaine puis <code>curl api.../api/settings</code>
E-mails non reçus	Logs <code>storage/logs/laravel.log</code> en mode log · sinon vérifier SMTP

Budget mensuel indicatif

POSTE	COÛT
Hostinger Unlimited (promo / renouvellement)	~\$3-4 / ~\$17-19
Bunny CDN + Storage	~\$1-3
Domaine .com (2 ^e année)	~\$15/an
Total au lancement	< \$8/mois

Récapitulatif express

Boutique : `darlila.com` · **Admin** : `darlila.com/admin` · **API** : `api.darlila.com/api` · **Maintenance** : `shop:down` / `shop:up` ·
Déploiement : `npm run deploy`